

Generative Models: VAE and GANs - Report

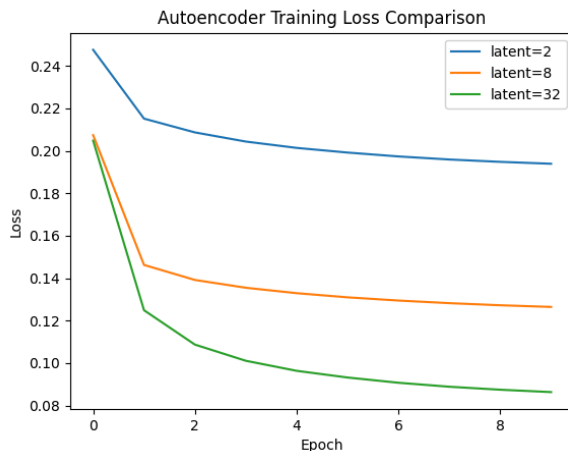
Justin Williams

10909201

Part 1: Variational Autoencoder (VAE)

Task 1 & 2:

Comparison:



Conclusion:

Task 1: Basic Autoencoder

A fully connected autoencoder was used with an encoder that compresses 784-dimensional MNIST images into latent spaces of size 2, 8, and 32. The decoder reconstructs the images back to the original size. The model uses ReLU activations, a sigmoid output layer, binary cross entropy loss, and the Adam optimizer. Results show that increasing latent dimensionality improves performance. The latent size of 2 produced the highest loss and worst reconstructions, while 32 gave the lowest loss and best image quality. The size of 8 performed in between. Overall, larger latent spaces preserve more information and reduce reconstruction error. Reconstruction quality follows the same trend. With a latent size of 2, outputs are blurry and heavily compressed. At 8, digits are clearer but still slightly distorted. At 32, reconstructions closely match the originals with minimal loss of detail. However, random sampling from the latent space produces poor results. Since the autoencoder does not enforce any structure on the latent distribution, sampled points often decode into noisy or unrecognizable images. This shows that the latent space is not suitable for generation.

Task 2: Variational Autoencoder (VAE)

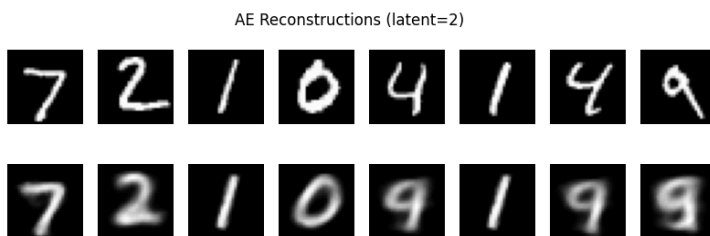
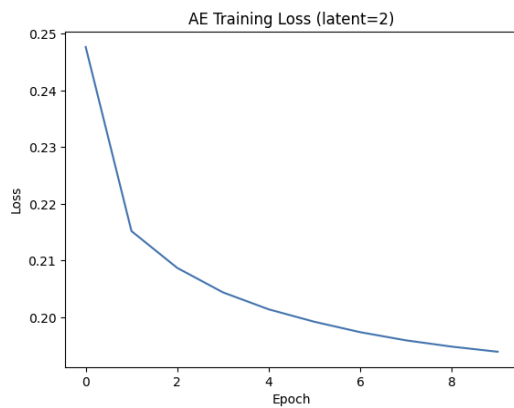
The VAE improves the autoencoder by making the latent space probabilistic. Instead of a single vector, the encoder outputs a mean and variance, and latent vectors are sampled using the reparameterization trick. The decoder then reconstructs the image from this sampled vector. The loss function includes both reconstruction loss (BCE) and KL divergence, which forces the latent space to follow a standard normal distribution. This regularization creates a smoother and more organized latent space. Training is slightly more complex due to the KL term, but it stabilizes over time. Unlike the AE, the VAE learns a structured latent space that supports meaningful sampling. Sampling results are significantly better than the AE. Random latent vectors produce clear and recognizable digits, and transitions between digits are smooth. This shows that the latent space is continuous and well structured. In the 2D latent visualization,

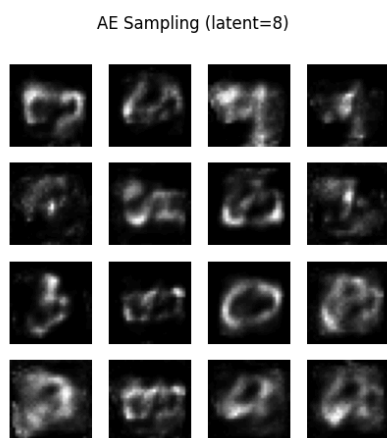
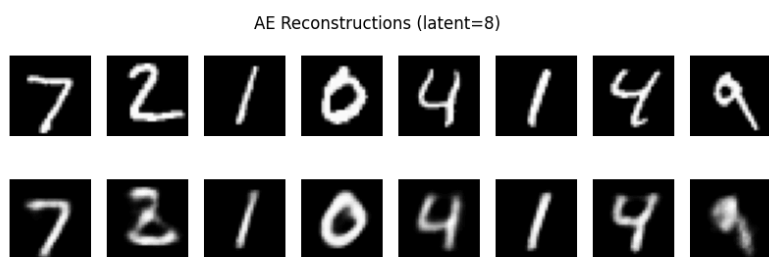
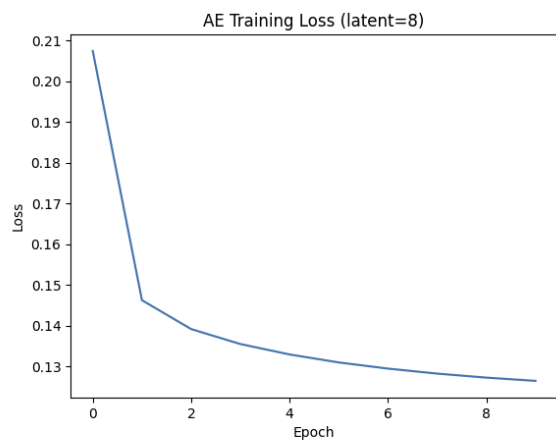
points near the center produce the best outputs, while extreme values produce lower quality images. This reflects the Gaussian structure enforced during training.

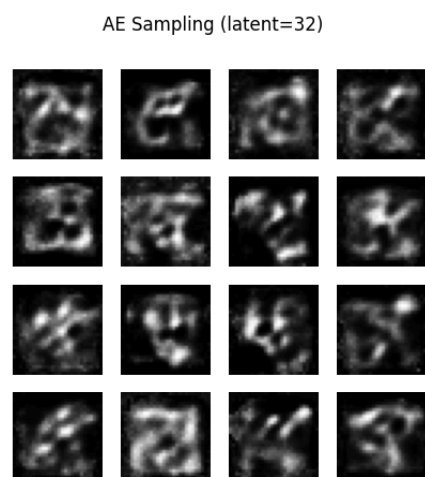
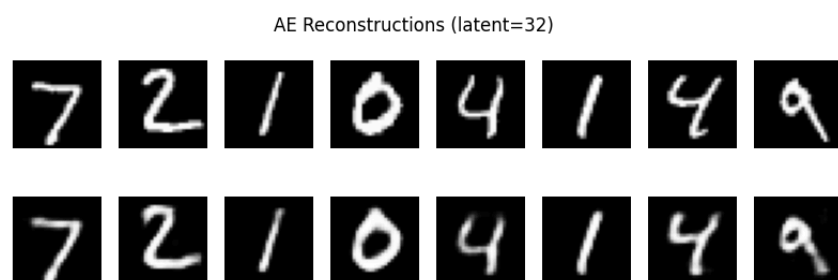
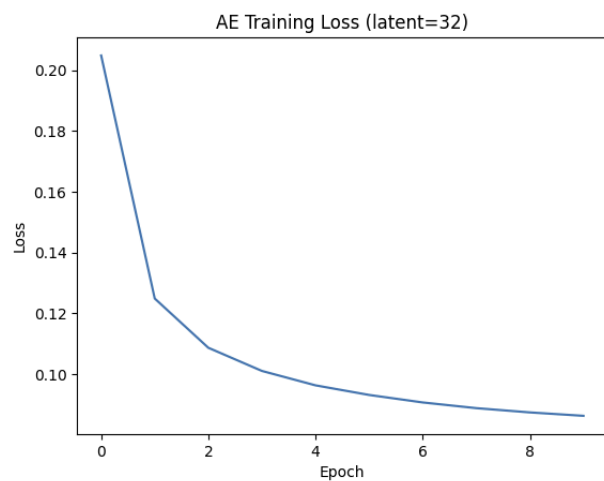
Conclusion:

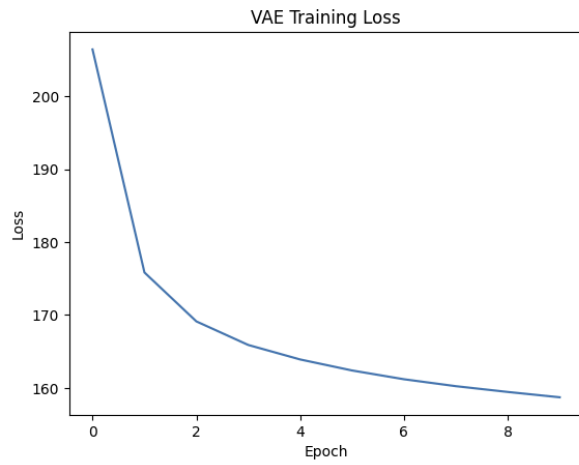
The autoencoder performs well at reconstruction but fails at generation due to an unstructured latent space. The VAE fixes this by enforcing a probabilistic distribution, enabling smooth sampling and meaningful generation.

Additional Attachments:









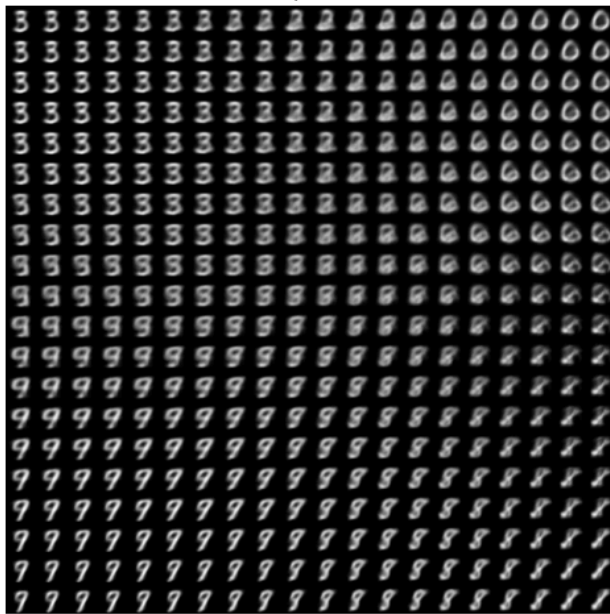
VAE Reconstructions



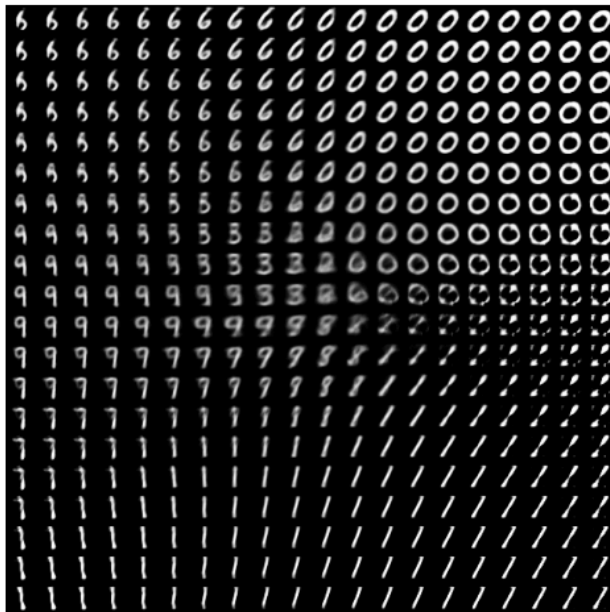
VAE Sampling



Latent Space (-1, 1)



Latent Space (-5, 5)



Part 2: Deep Convolutional GAN (DCGAN)

Task 1 & 2:

Conclusion:

Task 1: DCGAN Implementation

A Deep Convolutional Generative Adversarial Network (DCGAN) was implemented to generate synthetic Pokémon images of size $64 \times 64 \times 3$. The model follows the standard adversarial framework consisting of a generator and a discriminator trained simultaneously. The generator maps a random noise vector sampled from a latent space (dimension 100) into an RGB image. It is implemented using a sequence of transposed convolutional layers (ConvTranspose2d), each followed by batch normalization and ReLU activation. These layers progressively upsample the input from a low dimensional representation (1×1) to a full 64×64 image. A Tanh activation function is applied at the output layer to scale pixel values to the range $[-1, 1]$, which matches the normalized dataset. The discriminator is a convolutional neural network designed to classify images as real or fake. It consists of convolutional layers followed by batch normalization and Leaky ReLU activation functions. The use of Leaky ReLU helps mitigate the “dying ReLU” problem by allowing a small gradient when inputs are negative. The network progressively downsamples the input image until a single scalar output is produced, representing the likelihood that the input image is real. The model is trained using a binary classification loss, which improves numerical stability by combining the sigmoid operation with the loss function. During training, the generator learns to produce increasingly realistic images to fool the discriminator, while the discriminator improves its ability to distinguish real from generated images. This adversarial process continues for multiple epochs until a balance is reached.

Task 2: GAN Tuning and Evaluation

Replacing Leaky ReLU with standard ReLU in the discriminator resulted in noticeably less stable training. With standard ReLU, some neurons became inactive (outputting zero gradients), which slowed learning and occasionally caused the discriminator to stop improving. This negatively impacted the generator, leading to poorer image quality and less consistent outputs. In contrast, Leaky ReLU allowed small gradients for negative inputs, maintaining gradient flow and improving overall training stability and convergence. Several hyperparameters were explored to improve training performance:

- Learning rate (0.0002): This value provided a good balance between convergence speed and stability. Higher values caused instability, while lower values slowed training significantly.
- Batch size (64): A smaller batch size improved training speed and reduced memory usage, while still maintaining stable updates.
- Leaky ReLU slope (0.2): This value ensured sufficient gradient flow without overwhelming the learning process.

These choices are consistent with standard DCGAN recommendations and resulted in stable training behavior. The generator and discriminator losses exhibited the expected oscillatory behavior typical of GAN training. Unlike traditional models, GAN losses do not monotonically decrease. Instead:

- The discriminator loss fluctuated as it alternated between correctly and incorrectly classifying generated images.
- The generator loss also fluctuated, reflecting its ongoing attempt to fool the discriminator.

This dynamic indicates that both networks are learning simultaneously and competing in a minimax game. The absence of divergence or collapse suggests relatively stable training.

Generated images improved progressively over training:

- Early epochs (1–5): Images appeared as noisy blobs with little recognizable structure.
- Middle epochs (5–15): Basic shapes and color patterns resembling Pokémon characters began to emerge.
- Later epochs (15–20): Images became more coherent, with clearer outlines and more consistent features.

Saving generated samples every 5 epochs helped visualize this progression and demonstrated the model's ability to learn meaningful representations from the dataset. No severe mode collapse was observed during training. The generator was able to produce a variety of different Pokémon like images rather than repeating a single pattern. However, minor instability was present, as expected in GAN training, with occasional fluctuations in image quality and loss values. To improve robustness, architectural adjustments such as adaptive pooling in the discriminator were used to ensure consistent tensor dimensions and prevent runtime errors. This contributed to stable training across all epochs.

Conclusion:

The DCGAN model successfully learned to generate realistic Pokémon style images from random noise through adversarial training. The use of convolutional architectures, batch normalization, and Leaky ReLU activations contributed significantly to training stability and performance. Experimental results showed that proper hyperparameter selection and architectural design are critical for avoiding instability and achieving high quality outputs. Overall, the project demonstrates the effectiveness of GANs for image generation tasks, while also highlighting common challenges such as training instability and sensitivity to architectural design choices.